Animação com CSS

Por que usar?

- Melhoram a experiência do usuário.
- Guiam a atenção para elementos importantes.
- Deixam o site mais interativo.

Principais propriedades

- transition
- transform
- @keyframes + animation

TRANSITION

transition

- **Mudanças de valores de outras propriedades CSS de forma suave.**

- São animações reativas a eventos como `:hover`, `:focus`, etc.

- É possível animar **mais de uma propriedade ao mesmo tempo**

Propriedades de Transição

```
transition: [property] [duration] [timing-function] [delay];
```

Propriedade	O que faz	Exemplo
transition-property	Define qual propriedade será animada	transition-property: all;
transition-duration	Tempo da transição	transition-duration: 0.5s;
transition-timing-function	Curva de aceleração da transição	ease, linear, ease-in-out
transition-delay	Tempo para a animação começar	transition-delay: 0.2s;

`transition-timing-function`

- Define o ritmo da transição. Os mais usados:
  - ease → Começa devagar, acelera, e termina devagar
  - linear → Ritmo constante
  - ease-in → Começa devagar e acelera
  - ease-out → Começa rápido e desacelera
  - ease-in-out → Começa e termina devagar
  - cubic-bezier(...) → Ritmo personalizado

`transition-delay`

- Atraso antes da transição começar

- A transição só começa **após um tempo** em que o evento acontece.

TRANSFORM

`transform`

- Permite **aplicar transformações visuais em elementos HTML**, como **mover**, **girar**, **aumentar/diminuir**, **inclinar**, entre outras.

- A propriedade `transform` **não afeta o fluxo do layout.**

Principais funções

- translate(x, y)
- rotate(ângulo)
- scale(x, y)
- skew(x, y)
- matrix(a, b, c, d, e, f)

translate(x,y)

- Move o elemento no eixo x e/ou y

- Exemplo:

```
/* Move 50px pra direita e 20px pra baixo */
transform: translate(50px, 20px);
```

rotate(ângulo)

- Gira o elemento no sentido horário ou anti-horário

- Exemplo:

```
/* Gira 45 graus no sentido horário */
transform: rotate(45deg);

/* Gira 90 graus no sentido anti-horário */
transform: rotate(-90deg);
```

scale(x,y)

- Aumenta o diminui o tamanho do elemento.

- Exemplo:

```
/* Aumenta 20% em largura e altura */
transform: scale(1.2);

/* Mantém a largura, mas reduz a altura pela metade */
transform: scale(1, 0.5);
```

skew(x,y)

- Inclina o elemento nas direções horizontal e/ou vertical.

- Exemplo:

```
/* Inclina 20° no eixo X e 10° no Y */
transform: skew(20deg, 10deg);
```

`matrix(a,b,c,d,e,f)`

- Combina todas as anteriores num único valor (pouco usada diretamente).

@KEYFRAMES + ANIMATION

O que é o @keyframes

- Define os **quadros-chave** de uma animação em CSS.

- É **mudar de estilo ao longo do tempo**.

```
@keyframes nome-da-animacao {
  0%   { /* estilo inicial */ }
  50%  { /* meio da animação */ }
  100% { /* estilo final */ }
}
```

Propriedade `animation`

- Aplica a animação que foi definida com @keyframes.

```
.elemento {
  animation: nome-da-animacao 2s ease-in-out
infinite;
}
```

Propriedades

Propriedade	O que faz?
animation-name	Nome da animação (ligado ao @keyframes)
animation-duration	Duração da animação (ex: 2s, 500ms)
animation-timing-function	Curva de aceleração (ex: ease, linear, ease-in)
animation-delay	Espera antes de começar
animation-iteration-count	Quantas vezes repete (1, infinite, 3, etc.)
animation-direction	Direção da animação (normal, reverse, alternate, etc.)
animation-fill-mode	Mantém o estilo do início ou fim após a animação (forwards, backwards, both, none)

`animation-timing-function`

- linear
- ease (padrão)
- ease-in
- ease-out
- ease-in-out
- steps(n)
- cubic-bezier(...) (personalizado)

animation-direction

- normal: do início ao fim
- reverse: do fim ao início
- alternate: ida e volta (animação alterna)
- alternate-reverse: volta e ida

`animation-fill-mode`

- none: volta ao estilo original (padrão)
- forwards: mantém o estilo do último frame
- backwards: aplica o estilo do frame inicial durante o delay
- both: aplica os dois acima

`animation-iteration-count`

- 1, 2, 3...
- infinite: repete pra sempre

`animation-play-state`

- running: a animação está rodando
- paused: a animação está pausada (pode pausar via hover ou JS)

USANDO ::BEFORE E ::AFTER COM ANIMATION

`::before` e `::after` com `animation`

- As animações podem ser aplicadas aos pseudo-elementos da mesma forma que em elementos normais.

- A chave aqui é combinar ::before ou ::after com as propriedades de animação CSS, como **`@keyframes`**, **`transition`**, e **`transform`**.

CLIP-PATH

Clip-path

- É uma propriedade CSS que **“recorta”** um elemento, fazendo com que **só uma parte dele fique visível**.

- A parte visível é definida por uma **forma geométrica**, como:
  - circle() → círculo
  - ellipse() → elipse
  - inset() → tipo um retângulo com margens internas
  - polygon() → qualquer forma com múltiplos pontos

Animando...

- Pode **animar** os parâmetros do clip-path usando `transition` ou `@keyframes`

ANIMAÇÕES COM :CHECKED E :TARGET

Animações com `:checked`

- O seletor :checked é usado em **inputs do tipo checkbox ou radio**.

- Quando o input está **marcado (checked)**, ele aciona estilos específicos no CSS, permitindo **alterar propriedades visuais** ou **acionar animações** em outros elementos com ~ ou +.

O que você pode animar com `:checked`?

- Abrir/fechar menus
- Expandir cards
- Trocar temas (light/dark)
- Mostrar/esconder seções
- Animações com `transition` ou

Animações com `:target`

- `:target` é um seletor que se ativa **quando o elemento for o destino de um link** com âncora.

- O que dá pra animar?
  - Expandir descrições
  - Mostrar menus dropdown
  - Criar sliders de conteúdo
  - Estilizar elementos temporariamente